The Info Pharma Ordering System (IPOS) - Implementation Report

Centre for Software Reliability,
School of Informatics
City University,

London, 2026

Summary

This is the Implementation Report of the system created for the managing director of Info Pharma Ltd, by team 4, Readers of Systems Dependability at the Centre for Software Reliability, City University.

Authors

Herman Sildnes
Joanna Ayeni
Leon Seferi
Kanzah Hussain
Vladyslav Semanyshym
Yamin Ahmed
Jenis Khorshed

Distribution

The Managing Director of Info Pharma Ltd, the development team at the Centre for Software Reliability at the School of Informatics, City University London

Contents

1 Preface

1.1 Purpose and Scope of the document

This is the Implementation Report for one of the subsystems of IPOS, IPOS-SA, and the Implementation of the IPOS software, which includes IPOS-SA and two other subsystems, IPOS-CA and IPOS-PU. The document covers the Software compilation and run-time components of the system as well as testing plans and results from subsequent tests on the system. The document is provided in response to the initial statement of requirements by Info Pharma Ltd, the client, for whom we then implemented this system for.

The document explains various features of the subsystem explaining what they should do and how the two other subsystems will interact with it. This document conveys in detail (using fragments of code and testing results) to the client the expected behaviour of the IPOS-SA subsystems and especially how this subsystem will interact with the other subsystems of IPOS.

1.2 Intended audience and Reading Suggestions

The Managing Director of Info Pharma Ltd, the system analysts and architects at the Centre of Software Reliability (CSR), City University London

Some of the sections are intended for the CSR team only, which is clearly stated in the document.

1.3 History of the document

This is v2.0 of the document.

2 Introduction

IPOS will allow Info Pharma Ltd to automate their business process in three interrelated areas: i) merchant account management, ii) ordering products and iii) selling inventory to the general public. By introducing an online ordering system the merchants will be able to make pharmaceutical orders and subsequently sell these products in their shops.

The three areas of planned automation are expected to be covered by three subsystems: IPOS-SA, IPOS-CA and IPOS-PU, respectively. This document is focused on IPOS-SA. The other subsystems are covered in separate documents.

In addition to the services offered to merchants, IPOS-SA allows Info Pharma to maintain an inventory of products which merchants can then purchase. Thus allowing merchants to directly buy products and configure discount plans for their purchases and set up inventory for their own business.

IPOS-SA will also send invoices and reports of products bought and profit generated when requested by the merchant.

IPOS-SA will interact with the other subsystems, IPOS-CA and IPOS-PU, too.

3 Glossary

CSR	Centre for software reliability
IPOS	Info Pharma Ordering System

4 Anticipated Evolution of System Requirements

We expect that IPOS-SA will be deployed quickly. The operational experience from merchants and staff will inform the future decisions about the necessary changes. The envisaged likely changes to various parts of IPOS-SA are summarised below.

4.1 Validation

Currently the validation of account information provided during registration of their account will be checked by administrators in Info Pharma. In the future, however, we envisage that the validation may be done using 3rd party online services, which will require changes.

4.2 Enterprise SSO Login

In the future, a feature we envisage for the system is having an enterprise SSO log in where employees of a merchant can use their SSO authentication to log into the system and access company specific data and products.

5 Software Compilation / run-time components

5.1 Subsystem Set-up

IPOS-SA is implemented as a two-part web application, consisting of a REST API backend and single page application frontend. These are both designed to run locally on a single machine.

Backend-Python / FastAPI

The backend is written in Python 3.14+ using the FastAPI framework. Python is an interpreted language so requires no compilation step. Furthermore, the application is managed using uv, a Python package manager. This automatically resolves and installs all dependencies on first run.

The server can be started for the first time using the following:

cd backend
pip install uv
uv sync
uv run python seed_data_demo.py # populate with demo data
uv run uvicorn main:app –reload

The Uvicorn ASGI server (with FastAPI's standard install) handles all the HTTP requests. The database is SQLite in development – a single file (ipos_sa.db) is created automatically in the backend directory on the first run.

PostgreSQL is supported for production through the DATABASE_URL environment variable.

Frontend - JavaScript/React/Vite

The frontend is written in JavaScript (ES modules) using the React 19 library. It is served using Vite 7 – a development server and build tool. Additionally, Node.js v20 or higher is needed.

The frontend can be started for the first time using:

cd frontend
npm install
npm run dev

*Please note, to start application, open two terminals and start both backend and Frontend.

The application runs on http://localhost:5173

For production deployment, npm run build bundles the frontend into static files in frontend/dist/.

5.1.1 Files Created

Python is an interpreted language so no compiled binary files (.exe, .jar, .class) are produced. The runtime consists of source files executed directly by the Python interpreter. The equivalent of a compiled executable is the combination of the source files and virtual environment installed by uv.

File / Directory	Description	Location at Runtime
main.py	Entry point for application. Initialises the FastAPI app and registers all routers. It also configures CORS and creates all of the database tables upon startup.	backend/
pyproject.toml	Python configuration file. Declares all backend dependencies with version constraints (FastAPI, SQLModel, PyJWT etc) and the required Python version. Read by uv sync on first setup to install all packages into .venv/.	backend/
ipos_sa.db	SQLite database file that is created automatically on first run, when running seed data script. Contains the users, orders, products and invoices.	backend/
.venv/	Python virtual environment containing all installed packages (FastAPI, SQLModel, PyJWT, bcrypt). Created and managed by uv.	backend/.venv/
auth/, catalogue/, merchants/, orders/, reports/, commercial_applications/, audit/, core/	Feature modules where each contains models.py, router.py and service.py. These are executed by the Python Interpreter at runtime.	backend/<module>/
seed_data_demo.py	Script for populating the database with the official IPOS sample dataset prior to the demo. Run before starting backend.	backend/

package.json	Node.js project configuration file. Declares all frontend dependencies with version pins and defines the npm scripts (npm install, npm run dev, npm run build). Npm install reads this file and generates package- lock.json to lock exact resolved versions.	frontend/
frontend/dist/ (production only)	Static frontend files produced by npm run build. Contains index.html and hashed JS/CSS assets. Only needed if the Vite dev server is not used.	frontend/dist/

In development and demo mode, the Vite dev server serves the frontend directly from source so no build step is required.

5.1.2 source files

Uploaded on moodle as part of our project submission.

6 Deployment of run-time software components on hardware

6.1 List of the COTS (Commercial-off-the-shelf) software components

Backend runtime dependencies that are installed automatically by uv.

component	Version	Purpose
Python	3.14+	Interpreter / runtime environment
FastAPI	>= 0.128.5	Rest API web framework
Uvicorn	Standard - Bundled with FastAPI	HTTP Server
SQLModel	>= 0.0.32	ORM - database models and queries. Built on SQLAlchemy + Pydantic
Pydantic	>= 2.12.5	Request/response data validation and serialisation
Pydanic-settings	>= 2.12.0	Configuration management via environment variables
PyJWT	>= 2.11.0	JWT token generation and validation for authentication
bcrypt	>= 5.0.0	Password hashing
requests	>= 2.32.5	HTTP client for outbound calls to IPOS-PU

Frontend runtime dependencies installed by npm install.

Component	Version	Purpose
Node.js	V20+	JavaScript runtime / package manager host
React	19.2.0	UI component library
React DOM	19.2.0	DOM rendering for React

React Router DOM	7.13.0	Client side routing between pages
React Icons	5.5.0	Icon library (Feather icons used throughout)
Vite	7.3.1	Development server and production build tool

6.2 Deployment of run-time files on the target computer(s)

All runtime components of IPOS-SA are deployed on the same computer. No remote database server or remote application server is used. The SQLite database is a local file within the project directory. Both the backend API (localhost:8000) and the frontend (localhost:5173) run as local processes on the same machine.

The other IPOS subsystems (IPOS-CA and IPOS-PU) are also deployed on the same machine for integration testing purposes.

IPOS-SA communicates with IPOS-PU via a REST endpoint for membership application Notifications.

IPOS-CA communicates with IPOS-SA via the IPOS-SA REST API.

All inter subsystem communication occurs over localhost.

In conclusion, all runtime components of IPOS-SA (dependencies, database and other IPOS subsystems) are deployed on the same computer.

7 Testing plans and testing results

7.1 Unit testing of IPOS-SA

Unit tests are written using Vitest (the JavaScript equivalent of jUnit). Tests are located in tests/services/ and run with:

cd frontend
npm test -- --run

The apiClient is mocked in all unit tests so that no running backend is required.

Two service modules (the JavaScript equivalent of classes) are covered. Each module covers more than 5 methods. orderService.js implements OrderAPIImpl while authService.js implements LoginAPIImpl and staff account management.

7.1.1 Testing plan and results

orderService.test.js (OrderAPIImpl – Test Cases 1-27)

This wraps all order and catalogue API calls and exposes 6 functions **(placeOrder, trackOrderProgress, getOrderDetails, queryBalance, viewPreviousOrders, getCatalogue).** These are consistent with the class diagram generated in the first Deliverable.

*Test Case IDs are references to the created test cases from the first deliverable. Where N/A is written, this means it is additional coverage not specified in the first deliverable.

*Suite number references are a count of the total number of tests created. They can also be used as references to find the specific test in our source code under tests/services/.

placeOrder:

Suite:	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
1	SA-ORD-01	productID = 101 quantity = 10	Valid product, Valid quantity	Success = true, orderID and amountDue returned	pass
2	SA-ORD-02	productID = 200, quantity = 1	Product does not exist in inventory	Success = false, error returned	pass

3	SA-ORD-05	productID = 100, quantity = 0	Boundary value - quantity of zero	Success = false	pass
4	SA-ORD-06	productID = 100, quantity = 0	Negative quantity	Success = false	pass
5	SA-ORD-03	product ID = -1, quantity = 1	Invalid productID	Success = false	pass
6	SA-ORD-04	productID= 100, quantity = 1,000,000	Extremely large quantity, depends on stock	Outcome is boolean and both paths are handled	pass
7	N/A	productID = 101, quantity = 2	Checks if correct payload shape is sent to / orders	apiClient.post called with {items} array	pass

trackOrderProgress:

Suite:	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
8	SA-ORD-07	orderID = order-1 (valid, exists)	Is it a valid order in the system?	Returns status string e.g. ‘accepted’	pass
9	SA-ORD-08	orderID = 99999 (does not exist)	Order not found	Returns ‘unknown’	pass
10	SA-ORD-09	orderID = -1 (invalid)	Invalid order ID	Returns ‘unknown’	pass
11	SA-ORD-10	orderID = 0 (boundary)	Boundary value	Returns ‘unknown’	pass

getOrderDetails:

Suite:	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
12	N/A	orderID = ‘order-abc’ (valid)	Full order object returned, verifies all snake_case backend fields are converted to camelCase	merchantID = ‘merch-1’, orderDate = ‘2026-01-15’, discountAmount = 10.00, amountDue = 190.00, courierRef = ‘DHL12345’	pass
13	N/A	orderID = ‘order-1’ with items array	Order with line items returned. Verifies items are correctly mapped	items[0].productName = ‘Aspirin’, items[0].unitPrice = 8.00	pass
14	N/A	orderID = ‘bad-id] (non existent)	Order not found	Throws ‘order not found’ error	pass

queryBalance:

Suite:	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
15	SA-ORD-11	merchantID = ‘merch-202’ (valid, has outstanding balance)	Valid merchant with existing orders.	Returns creditLimit = 5000.00, currentDebt = 1200.00, availableCredit = 3800.00	pass
16	SA-ORD-12	merchantID=’ merch-202’ (valid, zero balance)	Valid Merchant with no outstanding orders.	Returns currentDebt = 0.00, availableCredit = 5000.00	pass
17	SA-ORD-13	merchantID = ‘9999’	Merchant not found.	Throws error	pass

		(does not exist)			
18	SA-ORD-14	merchantID = -1	Invalid merchant ID.	Throws error	pass
19	N/A	merchantID = ‘0’ (boundary value)	Boundary case – zero ID.	Throws error	pass

viewPreviousOrders:

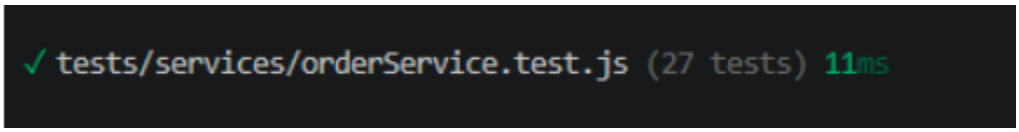
Suite:	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
20	SA-ORD-15	merchantID = ‘m1’, has 2 existing orders	Merchant exists and has placed orders.	Returns list of 2 order summaries.	pass
21	SA-ORD-16	merchantID = ‘m1’ but has no orders	Merchant exists but no order history.	Returns empty list.	pass
22	SA-ORD-17	merchantID = 2000 (does not exist)	Merchant not found.	Throws error	pass
23	SA-ORD-18	merchantID = -1 (invalid)	Invalid merchant ID	Throws error	pass
24	N/A	merchantID = ‘m1’, single order with all fields	Verifies camelCase Field mapping on List responses.	orderDate = ‘2026-02-01’, discountAmount = 15, amountDue = 285.	pass

getCatalogue:

Suite:	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome

25	SA-ORD-19	No parameters. Catalogue is pre-populated with 2 products.	Product exists in catalogue.	Returns list of 2 products. productCode = 'ASP001', packageCost = 8.50, minStockLevel= 50.	pass
26	SA-ORD-20	No parameters. Empty catalogue mock.	No products in catalogue.	Returns empty array[]. Array.isArray result is true.	pass
27	N/A	No parameters. Backend throws Error ('Server error').	Network or server error during fetch.	Returns [] instead of Throwing. Page still renders without crashing.	pass

orderService unit test result: 27/27 passed.



authService.test.js (LoginAPIImpl - Test Cases 28 - 56):

merchantLogin:

Suite:	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
28	SA-LOG-01	Username = "merchant1", password = "correctPassword".	Valid credentials	success = true, user returned, JWT stored via a setAuthToken	pass
29	SA-LOG-02	Username = "merchant1",	Wrong password	success = false, error	pass

		password = “wrongPassword”.		returned.	
30	SA-LOG-03	Username = “unknown”, password = “pass”.	Merchant does not exist	success = false, error returned.	pass
31	SA-LOG-04	Username = “”, password = “pass”.	Empty username	Success = false	pass
32	SA-LOG-05	Username = “merchant1”, password = “”.	Empty password	Success = false	pass
33	SA-LOG-06	Username = “”, password = “”.	Both fields empty	Success = false	pass
34	N/A	username = “merchant1”, password = “correctPassword”. Backend returns {message ‘ok’ } with no access_token field.	Verifies a response missing the token is treated as a failure	Returns success = false.	pass
35	N/A	username = “merchant1”, password = “pass” and the backend rejects.	Verifies any existing token is cleared before each login, even on failure	clearAuthToken called.	pass
36	N/A	username = “merchant1”, password = “pass123”.	Verifies the correct field names and values are sent to the login	POST to /auth/login with body { username: “merchant1”, password:	pass

			endpoint	“pass123” }.	

merchantDiscount:

Suite:	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
37	SA-LOG-07	(Valid session)	Merchant Logged in, logout requested.	Returns true and token cleared	pass
38	SA-LOG-08/09	(Backend errors)	Session not found or merchant does not exist.	Returns false and token still cleared locally	pass
39	SA-LOG-10	-	Correct logout endpoint hit	POST/auth/logout called	pass

getCurrentUser:

Suite:	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
40	N/A	No input parameters. Backend returns user object { id: ‘abc’, username: ‘admin1’ , role: ‘admin’, email: ‘admin@test.com’	Valid authenticated Session. Verifies pass through to /auth/me.	Returns user object with all fields. GET/auth/me called.	pass
41	N/A	No input parameters. Backend throws Error	Session expired or no token present.	Throws ‘Unauthorized error’.	pass

		(‘Unauthorized’).			

changePassword:

Suite:	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
42	N/A	currentPassword = ‘oldPass123’, newPassword = ‘newPass456’	Valid passwords, Correct current Password supplied.	Returns success =true.	pass
43	N/A	currentPassword= ‘wrongOldPass’, newPassword = ‘newPass456’.	Wrong current Password supplied.	Returns success = false, error Message contains ‘incorrect’.	pass
44	N/A	currentPassword= ‘oldPass123’, newPassword = ‘123’ (only 3 characters).	New password too short.	Returns success = false.	pass
45	N/A	currentPassword= ‘oldPass’, newPassword = ‘newPass’.	Verifies correct Snake_case field names sent to backend.	POST to /auth/change-password with Body {current_password, new_password}. Not camelCase.	pass

getStaffUsers:

Suite:	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
46	N/A	No input parameters Backend returns list of 2 users with is_active field.	Staff list retrieved Verifies snake_case To camelCase mapping on List responses.	Returns list of 2 users users[0].isActive = true, users[0].username='admin1'.	pass
47	N/A	No input parameters. Backend throws Error ('Forbidden).	Unauthorized role attempts to access staff list.	Throws 'Forbidden' error.	pass

createStaffUser:

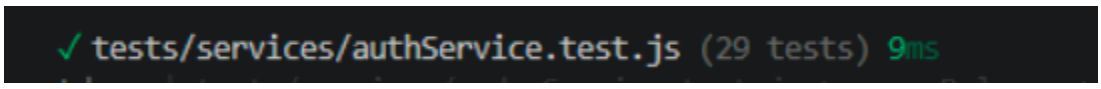
Suite:	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
48	N/A	username = 'manager2', email = 'mgr@test.com', password = 'Pass123, role = 'manager'.	Valid new staff user details.	Returns success = true, Result.user.username = 'manager2'.	pass
49	N/A	username = 'admin1' (already exists), email = 'taken@test.com',	Username already in use.	Returns success = false, error returned.	pass

		password = ‘pass’, role = ‘admin’.			

changeUserRole:

Suite:	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
50	N/A	userId =’2’, role = ‘director’.	Valid role change for an existing user.	Returns success = true, result.user.role = ‘director’.	pass
51	N/A	userId = ‘self-id’, role = ‘manager’.	Attempting to change own role. Backend prevents self-demotion.	Returns success = false.	pass
52	N/A	userId = ‘5’, role = ‘admin’.	Verifies correct endpoint and request body.	PATCH /auth/users/5/role called with body {role: ‘admin’ }.	pass

authService unit test result: 29/29 passed.



Overall unit test result: 56/56 passed.

7.2 Subsystem testing with IPOS-PU and IPOS-CA

7.2.1 Subsystem Testing - Provided Interface Test

IPOS-SA provides two interfaces used by external subsystems:

ISALoginAPI is used by IPOS-CA to authenticate merchants.

ISAOrderAPI is used by IPOS-CA to place and track orders. One method from each provided interface is tested.

ISALoginAPI.merchantLogin in merchantLogin.test.js (Suites 57-68):

Suite:	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
57	SA-LOG-01	username = “merchant1”, password = “correctPass” .	Valid credentials	success = true, user object returned.	pass
58	SA-LOG-02	username = “merchant1”, password = “wrongPass”.	Wrong password	success = false, error returned.	pass
59	SA-LOG-03	username = “unknown”, password = “pass”.	Merchant does not exist.	success = false, error returned.	pass
60	SA-LOG-04	username = “”, password = “pass”	Empty username	success = false, error returned.	pass

61	SA-LOG-05	username = “merchant1”, Password = “”.	Empty password	success = false, error returned.	pass
62	SA-LOG-06	username = “”, password = “”.	Both fields empty	Success = false, error returned	pass
63	N/A	username = “merchant1”, password = “correctPass” .	Verifies that the correct HTTP Method and Endpoint are used.	POST request sent to /auth/login exactly once.	pass
64	N/A	username = “merchant1”, password = “correctPass”	Verifies the request body uses the correct field names	POST body contains { username: “merchant1” , password: “correctPass” }	pass
65	N/A	username = “merchant1”, password = “correctPass” .	Verifies the JWT token returned in the response is stored.	setAuthToken called with ‘valid-jwt-token’.	pass
66	N/A	username = “merchant1”, password = “correctPass” (backend rejects).	Verifies any Existing token is Cleared before each login attempt.	clearAuthToken called before the request.	pass
67	N/A	username = “merchant1”, password = “wrongPass”.	Verifies no token is stored when login fails.	setAuthToken not called.	pass

68	N/A	username = “merchant1”, password = “correctPass” , Response body contains no access_token field.	Verifies a response missing the token field is treated as a failure.	Returns success = false and setAuthToken is not called.	pass

Result: 12/12 passed.

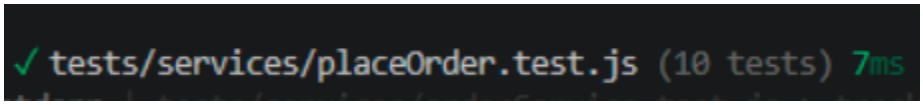
ISAOrderAPI.placeOrder in placeOrder.test.js (Suites 69-78):

Suite:	Test Case ID	Parameter Values	Description	Expected Outcome	Actual Outcome
69	SA-ORD-01	productID = 1001, quantity = 5	Valid product, Valid quantity.	success = true	pass
70	SA-ORD-02	productID = 9999, quantity = 5	Product does not exist.	success = false	pass
71	SA-ORD-05	productID = 1001, quantity = 0	Boundary value – zero quantity.	success = false	pass

72	SA-ORD-06	productID = 1001, quantity = -1	Negative quantity.	success = false	pass
73	SA-ORD-03	productID = -1, quantity = 5	Invalid product ID.	success = false	pass
74	SA-ORD-04	productID = 1001, quantity = 1000000	Extremely Large quantity.	Outcome is boolean regardless of stock level.	pass
75	N/A	productID = 1001, quantity = 5	Verifies the correct HTTP Method and Endpoint are used.	POST request sent to /orders exactly once.	pass
76	N/A	Items = [{ productid:1001, quantity:5}]	Verifies the items array is sent in the request body.	POST body contains { items: [{ productid: 1001, quantity: 5 }] }	pass
77	N/A	productID = 1001, quantity = 5. Backend returns order_id = ‘ORD-001’, amount_due = 90, discount = 10.	Verifies snake_case Response fields are correctly mapped to camelCase.	result.orderId = ‘ORD-001’, result.amountDue = 90	pass
78	N/A	productID = 1001,	Verifies a network error	Returns success	pass

		quantity = 1. Backend throws Error ('Network error').	returns a failure object and does not throw.	= false. Does not throw.	

Result: 10/10 passed



Overall subsystem test result: 22/22 passed.

7.2.2 System Testing - Required Interface

IPOS-SA uses IPUCommsAPI provided by IPOS-PU to send email notifications to applicants following a commercial membership decision. The method under test is sendEmail(email, body, subject). This is accessed via the notifyApplication wrapper in notificationService.js in frontend/services/.

The brief requires tests for 2 methods of the required interface. IPUCommsAPI exposes only a single method: sendEmail(email, body, subject). As this is the only method available on the required interface, all system tests cover this method.

Results are not required and can only be obtained during integration testing with the IPOS-PU team.

notifyApplication.test.js:

Suite:	Test Case ID	Parameter Values	Description	Expected Outcome
79	SA-IPU-01	Email = application@test.com, valid outcome = 'approved' , valid company ref.	All valid parameters.	Returns true. POST made to IPOS-PU with correct email, Correct subject containing outcome, as

				well as a body Containing the company reference.
80	SA-IPU-03	Outcome = ‘pending’ (non-terminal state).	Pending – no email should be sent.	Returns false Without calling sendEmail.
81	SA-IPU-04/05	IPOS-PU unavailable (connection refused)	Required interface unavailable.	Returns false. An attempt was made but did not crash.
82	SA-IPU-06	email = “” , outcome = ‘pending’.	Completely invalid input	Returns false Without calling sendEmail.

Overall test suite result:

82/82 passed

*Please note, upon running the test suite, stderr lines are visible in the output. This is expected and they are internal error messages logged by the service functions when error handling paths are used by the mocked responses. They do not indicate test failures. All 82 tests pass.

7.3 Non-Functional Testing Considerations

The following describes how nonfunctional attributes of a production deployment of IPOS-SA could be tested. Actual results from these tests are not provided as they are not required at this stage.

Security:

IPOS-SA exposes a login endpoint at POST /auth/login that accepts username and password credentials. A brute force test should be conducted to verify that the backend correctly rate limits or rejects repeated failed attempts from the same origin (within a short time window).

Additionally, all API endpoints that accept user supplied string input (such as order placement and merchant queries) should be tested for injection resistance. This risk is already reduced by SQLModel's use of parameterised queries internally, but the endpoints should be verified under automated security scanning.

JWT token expiry should be confirmed by attempting to use a token beyond its configured lifetime and verifying a 401 response is returned.

Scalability:
Based on the staff structure and the number of registered merchants, realistic peak concurrent usage is estimated at 10-20 simultaneous users. A load testing tool like Locust or k6 could simulate 10 times this load to verify the system remains responsive. Acceptable thresholds would be under 500ms for write operations and under 200ms for read operations at peak loads. If write performance deteriorates, this would indicate the need to switch from the SQLite development configuration to the PostgreSQL production setup.

Reliability and Recoverability:
The backend can be tested for correct state recovery by forcibly terminating the uvicorn process mid request during an order placement and then restarting it. Since SQLite only commits fully completed transactions, the database should reflect only the last committed state with no partial writes. For a production PostgreSQL deployment, this same principle would also apply. This would also involve verifying connection pool recovery on restart. To become reasonably confident in runtime reliability, we'd need to achieve continuous automated execution of the full test suite over several hours. This is with the assumption that no failures or state corruption is observed.

8 Appendicies

8.1 UML Design class diagram derived from final source code

[paste diagram here]

8.2 Initial Statement of requirements by the client

IPOS: Info Pharma Ordering System

The InfoPharma Ordering System (IPOS) will keep track of the orders of pharmaceutical goods made by merchants to InfoPharma Ltd and will assist the merchants' sales to consumers.

IPOS includes ***three*** subsystems:

I. A server application (IPOS-SA), which in the fully functional production version will be administered by the IT department of InfoPharma Ltd.
II. A client application (IPOS-CA), a copy of which will be run by the merchant on a desktop PC in the merchant's Pharmacy shop.
III. An online platform (IPOS - PU) to be used by a merchant for direct sales of pharmaceutical goods to the public. The platform will allow the merchants to run advertisement/promotion campaigns with new products with discounts.

In the prototype of IPOS the three subsystems should be implemented as separate ***desktop applications*** able to interact with one another as necessary to meet the requirements. For instance, online sales (via IPOS - PU) should be linked to the inventory of the merchant: successful online purchases should be deducted from the stock of goods available at merchant's shop. Likewise, orders from IPOS-SA and delivered to a merchant, should be easy to record in IPOS-CA and increase the merchant's stock. Finally, promotion campaigns run via IPOS – PU may affect the cost of selling goods online (at reduced price), but not the cost of the same goods if sold in the pharmacy shops.

8.3 Hardware (minimal and optimal configuration)

8.3.1 minimal hardware configuration

The minimal hardware configuration recommended for IPOS development consists of a desktop PC, 1GB of RAM and at least 50GB of disk space for storing the software programs and data possibly running a database server such as MySQL or other similar. A hosting package with preinstalled software and at least 1 GB for use data is also acceptable for both development and live deployment of IPOS.

8.3.2 optimal hardware configuration

The optimal hardware configuration envisaged for development should consist of a desktop PC, 4GB of RAM and at least 250GB of disk space for storing the software programs and data possibly running a database server such as PostgreSQL or better. Again, a hosting package with preinstalled software + 5GB of data is considered an optimal hardware configuration for both development and live deployment

8.4 AI Declaration

Claude (Anthropic) Sonnet 4.6 was used during the writing of Section 7.3 (Non- Functional Testing Considerations). Before writing, we were unsure of which categories of non-functional testing were most relevant to cover.

Prompt:
*we provided Claude Sonnet 4.6 with prior sections and then asked the following.

"We are writing the non-functional testing section. What categories of non-functional testing are worth considering? Don't write any content, just list the areas and what each one generally focuses on."

Claude Sonnet 4.6 Response:

"Here are the main non-functional testing categories worth considering:

1. Security — testing authentication endpoints against attacks, verifying injection resistance, and confirming token/session expiry behaviour
2. Performance/Scalability — load testing under concurrent users, measuring response times, identifying bottlenecks
3. Reliability and Recoverability — testing system behaviour after unexpected failures, verifying data integrity is maintained
4. Availability — measuring uptime and failover behaviour under sustained use
5. Usability — assessing how easily the interface can be used by the intended users
6. Maintainability — how easily the codebase can be modified, extended, or debugged

For a backend-heavy system with a database, Security, Scalability, and Reliability tend to be the most substantive to discuss. "

From these, I selected Security, Scalability and Reliability and Recoverability. All specific descriptions within each category were written by me based on my own knowledge of the system but were guided and influenced by Claude's various suggestions.

9 Index

Not provided in this release of the document